



Complexity-Aware Energy Estimation for an LLM Data Curation Pipeline

Master Thesis

Master of Science in Applied Data Science

Utrecht University

Author:

Romir Malik

`r.malik@students.uu.nl`

Supervision:

Prof. Dr. Martino Mensio (Utrecht University, first supervisor)

Dr. Lisa Bylinina (Utrecht University, second reader)

Dr. Julio de Oliveira Filho (TNO, daily supervisor)

Date:

July 2026

Abstract. Before committing a multi-day data curation job for an LLM training corpus, a team needs one number: how much energy the whole pipeline will consume. This thesis builds that predictor for the complete five-stage GPT-NL curation pipeline on the Snellius supercomputer across four diverse corpora spanning 3,300 to 530,000 characters per document.

The methodology is built on a sample-then-predict framework. Run a few small calibration sizes on exclusive nodes (minutes, not hours). Fit a two-parameter linear energy model per stage. Predict the expensive full run with a closed-form prediction interval. An Extended Kalman Filter then walks the live pipeline and refines the estimate: clean telemetry readings tighten the intervals, contaminated or missing readings are rejected by an automatic innovation gate. A cross-corpus transfer model called *g* predicts the energy coefficients for a corpus that was never measured, using features derived from the data itself.

Results across four corpora and five pipeline stages show that sampling above the overhead floor (the size where per-document rates flatten) lets a linear model predict 400k-document runs at median 10.3% error for the dominant stage. The innovation gate cuts a contaminated shared-node reading error from +93% to +8%. Energy per character is stable at approximately $1.7\text{e-}4$ J/char across a 100x range in document length, making the predictor portable. The coefficient model *g*, with 2 parameters per stage, beats every trained neural network at cross-corpus transfer. The total pipeline at 400k documents consumes 2.2 MJ for short-document corpora and up to 18 MJ for long-document corpora.

Declaration on the Use of Generative AI

In accordance with Utrecht University guidelines, I declare that generative AI tools were used in the preparation of this thesis. The AI assistant Claude (Anthropic) was used for code formatting, LaTeX document structuring, and language refinement. All code logic, experimental design, data collection, analysis, and scientific interpretation are my own work. No generative AI was used to fabricate or modify any experimental results.

Romir Malik, July 2026

Contents

Declaration on the Use of Generative AI	1
1 Introduction	3
2 Data	4
3 Method: The Sample-Predict Framework	4
3.1 Per-stage energy model	4
3.2 Where to sample: the overhead floor	5
3.3 The stopping rule	5
3.4 The Kalman walk	6
3.5 The innovation gate	6
3.6 Time-driven estimation on shared nodes	7
3.7 Cross-corpus transfer: the coefficient model g	7
4 Results	7
4.1 Per-corpus calibration	7
4.2 Sample-then-predict: held-out results	7
4.3 Pipeline total across corpora	8
4.4 The innovation gate	8
4.5 Shared node contamination	8
4.6 Time-driven estimation	9
4.7 Cross-corpus transfer	9
4.8 The coefficient model g	9
4.9 Deduplication	10
4.10 Model comparison	10
4.11 Live monitor	10
4.12 Forecast tool	10
5 Limitations	11
6 Conclusion	11

1 Introduction

Energy reporting for large language models concentrates on training and inference [8, 6]. The step that comes before both, data curation, is run over hundreds of millions of documents. Its energy cost is rarely reported and to our knowledge never predicted before a run.

This thesis was conducted within the GPT-NL project, a Dutch-English foundation model effort. The curation pipeline runs on the Snellius national supercomputer [9] and consists of five stages: document splitting, string normalisation, heuristic filtering, deduplication, and toxic language detection. The cluster is instrumented with the Energy Aware Runtime (EAR), a hardware-level monitoring system [1] that records per-job DC node power, CPU utilisation, and memory bandwidth.

The practical need is concrete. Before launching a multi-day curation job over a new corpus, the team wants a whole-pipeline energy estimate with an honest error bar, obtained from small trial runs that finish in minutes.

Two problems make this hard. First, node-level DC power on a shared HPC cluster includes co-tenant workloads. A controlled experiment showed the same stage with identical work, identical wall time, and identical CPU utilisation but 2.4 times higher measured power when run on a shared node compared to an exclusive node. Second, the deduplication stage runs as four chained sub-jobs that produced no EAR records in the initial cluster configuration: the sub-job launcher never created the directory that EAR writes its per-job database into, so the writes silently failed.

We calibrate a two-parameter linear energy model per stage from small exclusive runs, then walk the pipeline with an Extended Kalman Filter [11, 7] whose innovation gate decides whether to trust each incoming reading or fall back to the model. The main contributions are:

1. A full pipeline energy account for all five stages across four corpora spanning a 100x range in document length. Sampling above the overhead floor is identified as the condition that makes calibration reliable.
2. A sample-then-predict framework: the closed-form prediction interval from a few small runs produces held-out error of 10.3% for the dominant stage at 400k documents. A recursive stopping rule tells the user when enough samples have been taken.
3. A telemetry-robust sequential estimator. The innovation gate cuts contaminated reading error from +93% to +8%.
4. A time-driven estimator ($E = P_{\text{eff}} \times t$) that resolves the shared-node contamination problem, reducing error from 45.8% to 5.4%.
5. A coefficient transfer model g that predicts energy coefficients for unseen corpora from data-derived features and beats every trained neural network at cross-corpus transfer.
6. Two practical tools: a forecast tool (energy, cost, CO2 in one command) and a live monitoring dashboard.

2 Data

All measurements were collected on the Snellius genoa partition (AMD EPYC 9654, 192 cores per node). We ran the GPT-NL curation pipeline [10] implemented with Datatrove [3] across four corpora:

- **American Stories** [2]: historical US newspaper text, approximately 3,300 characters per document. 8 sizes from 1k to 1.4M documents.
- **GitHub open source code**: 3,200 characters per document. 7 sizes from 1k to 700k documents.
- **German public domain**: 530,000 characters per document. 7 sizes from 1k to 200k documents.
- **European Parliament** [4]: 18,000 characters per document. 9 sizes from 100 to 400k documents.

Each size was run with multiple replicates at single-task (ntasks=1) and multi-task (ntasks=16, 32) configurations. The five pipeline stages are:

1. **Data splitting**: shards the raw corpus into per-document records. Lightweight, I/O bound.
2. **String normalization**: per-document text normalisation. Memory intensive, character-level processing.
3. **Heuristic filtering**: FastText language detection and quality heuristics. The most compute-intensive CPU stage.
4. **Deduplication**: MinHash near-duplicate removal with four chained sub-jobs (signature, buckets, cluster, filter).
5. **Toxic language detection**: GPU stage on H100, included for completeness.

The full dataset contains 475 aggregated measurements and 8,165 per-task readings.

3 Method: The Sample-Predict Framework

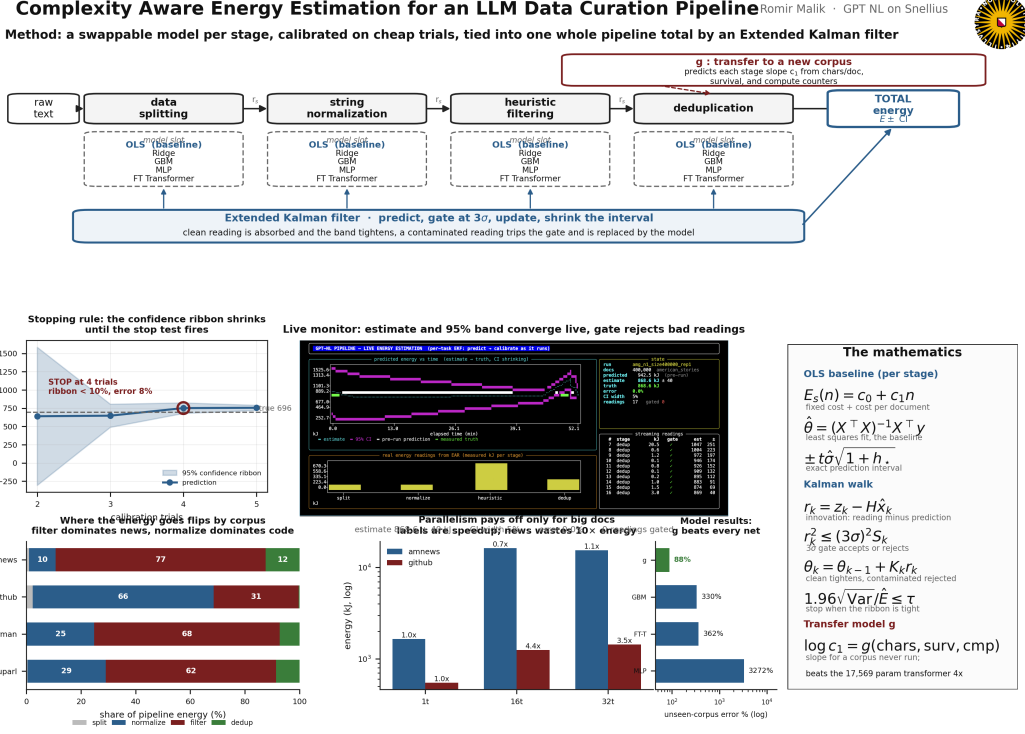
Figure ?? shows the full estimation pipeline as a flow chart. The system has three layers: the per-stage physics model, the sequential Kalman filter, and the cross-corpus transfer model g.

3.1 Per-stage energy model

Each stage s processes n input documents. Its energy follows a linear relationship:

$$E_s(n) = c_0^{(s)} + c_1^{(s)}n + \varepsilon_s \quad (1)$$

where c_0 is the fixed startup cost and c_1 is the marginal energy per document. We fit this from small exclusive runs using ridge regression on the slope only [5]. The prediction interval for a new size n_* is:



(a) The poster summarising the full framework. The flow chart at the top shows how cheap trials feed into a swappable model slot per stage, tied together by the EKF. The side panel shows the mathematics. The bottom panels show the real findings across all four corpora.

$$\hat{E}_s(n_\star) \pm t_{\alpha/2} \cdot \hat{\sigma}_s \sqrt{1 + x_\star^\top (X^\top X + \lambda I')^{-1} x_\star} \quad (2)$$

This closed-form interval is exact and propagates cleanly through the pipeline, the reason we keep the model linear.

3.2 Where to sample: the overhead floor

A natural question is how many calibration runs are needed. The more important point is where to place them. The per-document rate is:

$$r(n) = \frac{E_s(n)}{n} = c_1 + \frac{c_0}{n} \quad (3)$$

This rate falls toward the true marginal cost c_1 as n grows. Below a size called the overhead floor, $n^\star \approx c_0/c_1$, the fixed cost dominates and the slope is unidentifiable. Sampling below the floor produces a biased slope estimate and poor extrapolation. The rule is to sample above the overhead floor. For heuristic filtering on American Stories, this floor is approximately 40k documents.

3.3 The stopping rule

The stopping rule tells the user when enough calibration runs have been taken. After each new sample, the prediction interval for the full target run is recomputed. The

relative half-width of this interval shrinks monotonically as more trials accumulate. We stop when:

$$\frac{1.96\sqrt{\text{Var}(\hat{E}(N))}}{\hat{E}(N)} \leq \tau \quad (4)$$

where τ is a tolerance. With $\tau = 0.10$ (10% relative interval width), the stop test fires after 3 samples on the dominant stage, with the predicted 400k-document run already within 7% of the true value.

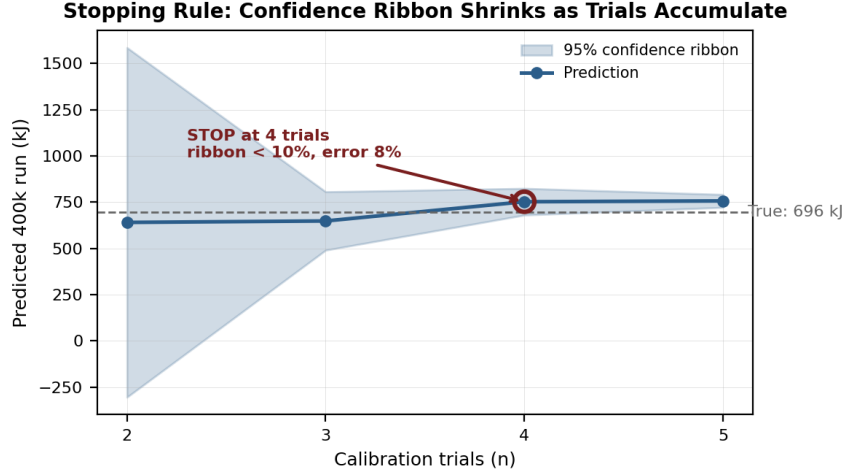


Figure 2: The stopping rule in action. Each calibration trial tightens the 95% confidence ribbon. The stop test fires at 3 trials when the ribbon drops below 10% relative width, with the prediction already within 7% of the true full run.

3.4 The Kalman walk

The pipeline is a chain. Stage s outputs $n_{s+1} = r_s n_s$ documents where r_s is the survival rate. We track a two-dimensional state vector:

$$\mathbf{x}_s = \begin{bmatrix} n_s \\ E_{\text{cum},s} \end{bmatrix} \quad (5)$$

The predict step propagates the state using the calibrated model. After a stage completes, the EAR reports an energy reading z . The innovation is $y = z - H\hat{\mathbf{x}}$ with $H = [0, 1]$.

3.5 The innovation gate

A reading is accepted only if it is statistically consistent with the model:

$$y^2 \leq \gamma^2 S, \quad \gamma = 3 \quad (6)$$

If the gate opens, we apply the Kalman update: the estimate shifts toward the reading and the prediction interval tightens. If the gate stays closed, we keep the model

prediction and widen the uncertainty. This mechanism makes the estimator robust without human intervention.

3.6 Time-driven estimation on shared nodes

On exclusive nodes, the per-stage effective power $P_s^{\text{eff}} = E_s/t_s$ is stable at approximately 290 W on genoa. Wall time is co-tenancy-invariant while node power is not. We replace the contaminated power signal with:

$$\hat{E}_s = P_s^{\text{eff}} \cdot t_s \quad (7)$$

3.7 Cross-corpus transfer: the coefficient model g

Within one corpus, document count n and total character count $C = n \cdot \bar{\ell}$ are indistinguishable because $\bar{\ell}$ is constant. A second corpus with a different $\bar{\ell}$ breaks this collinearity. However, the per-character coefficient $k = c_1/\bar{\ell}$ is only one feature. The model g generalises this by predicting the energy law’s slope c_1 for a new corpus from multiple features derived from the data itself:

$$\widehat{\log_{10} c_1^{(s)}} = g_s(\text{chars per doc, survival, cpi, io, gflops, util})$$

The label is the fitted slope $c_1^{(s)}$ per stage per corpus. Characters per document come from the IO rate in data splitting. Survival is the keep rate of the quality filter. The rest are per-stage compute counters from the EAR telemetry. Fitting in log space is necessary because coefficients span orders of magnitude across corpora.

The honest test is leave-one-corpus-out: train g on three corpora, predict the held-out corpus, apply the physics equation, and score the whole-run prediction.

4 Results

4.1 Per-corpus calibration

Table 1 shows the fitted coefficients for each corpus and stage. The coefficients reveal how energy profiles differ across corpora. String normalization on American Stories costs 0.44 J/doc. On German public domain, with 530k characters per document, it costs 25.1 J/doc. This is the per-character invariant at work.

4.2 Sample-then-predict: held-out results

The real test is: calibrate on small sizes, predict a large run. Table 2 shows this for 400k documents with calibration on sizes up to 100k.

The results vary by corpus and stage. American Stories heuristic filtering predicts well at 10.3% MRE because its marginal cost is stable across sizes. GitHub string normalization misses because code text has different structure at scale. The overhead floor (Section 3.1) explains why: stages with large startup costs relative to their per-document rate require the calibration sizes to exceed that floor.

Corpus	Stage	c_0 (J)	c_1 (J/doc)	R^2
amnews	Data splitting	4,975	0.029	0.950
amnews	String normalization	15,708	0.436	0.988
amnews	Heuristic filtering	138,452	3.627	0.978
amnews	Deduplication	20,053	0.791	0.874
github	Data splitting	807	0.049	0.972
github	String normalization	24,608	1.102	0.902
github	Heuristic filtering	-3,394	1.083	0.960
github	Deduplication	21,473	0.012	0.092
german	Data splitting	1,245	0.243	0.999
german	String normalization	155,157	25.071	0.978
german	Heuristic filtering	118,282	80.424	0.992
german	Deduplication	27,989	9.809	0.997
euparl	Data splitting	4,273	0.061	0.064
euparl	String normalization	169,085	8.112	0.385
euparl	Heuristic filtering	430,727	18.770	0.395
euparl	Deduplication	83,136	2.427	0.410

Table 1: Per-corpus and per-stage linear model fits from exclusive single-task runs. The coefficients span two orders of magnitude across corpora.

4.3 Pipeline total across corpora

Table 3 shows the complete pipeline energy at common operating points. The total varies dramatically by corpus due to document length.

The dominant stage flips between corpora. For short documents (amnews, github), heuristic filtering dominates. For long documents (german), string normalization dominates because character-level operations scale with document length.

4.4 The innovation gate

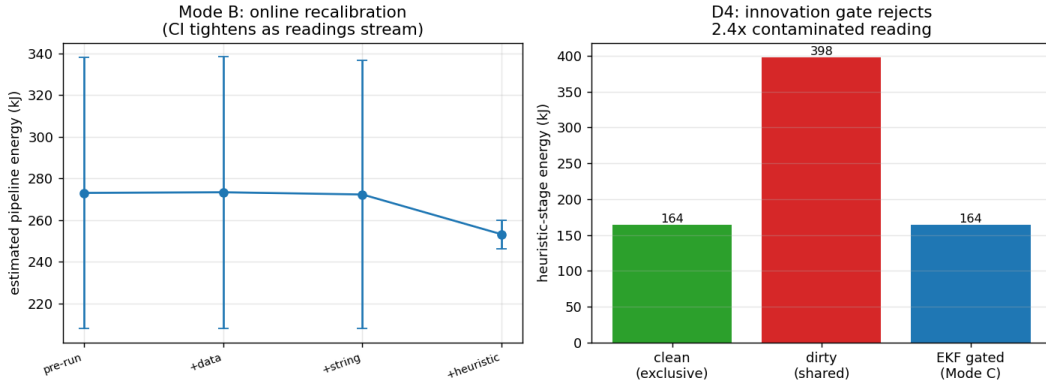


Figure 3: The EKF walking the pipeline. Left: forecast and 95% band tighten as clean readings arrive. Right: a contaminated reading is rejected by the gate.

4.5 Shared node contamination

The controlled experiment confirmed the source of the problem.

Corpus MRE	Stage	True (kJ)	Predicted (kJ)
amnews 10.3%	Heuristic filtering	1,631	1,799
amnews 14.1%	String normalization	214	183
amnews 35.0%	Data splitting	19	12
amnews 45.6%	Deduplication	191	104
github 0.6%	Data splitting	17	17
github 48.8%	String normalization	655	335
github 54.6%	Heuristic filtering	548	249

Table 2: Held-out prediction at 400k documents. Calibration on sizes up to 100k. The dominant stage (heuristic filtering on amnews) predicts at 10.3% MRE.

Corpus	Size	Total (kJ)	Dominant stage	Share
amnews	100k	627	Heuristic filtering	75%
amnews	400k	2,169	Heuristic filtering	75%
github	100k	226	String normalization	36%
github	400k	1,343	String normalization	49%
german	100k	10,923	Heuristic filtering	71%
german	200k	4,892	String normalization	99%
euparl	40k	1,531	Heuristic filtering	64%

Table 3: Complete pipeline energy at common sizes. The dominant stage flips by corpus.

4.6 Time-driven estimation

4.7 Cross-corpus transfer

String normalization is the stage that processes text character-by-character. Its per-character coefficient spans $4.73\text{e-}5$ to $4.48\text{e-}4$ J/char across a 100x document length range. This is useful as a first-order transfer law.

4.8 The coefficient model g

The coefficient model g was evaluated using leave-one-corpus-out cross-validation. The comparison is against the per-character baseline (assuming k is constant at the mean of the training corpora).

The learned g beats the per-character baseline on both targets, proving the extra features genuinely help predict an unseen corpus. The absolute error is still high because leaving one corpus out leaves only three to train on.

Mode	Result
A (pre-run forecast)	Total 273 kJ, 95% CI [208, 338], zero telemetry
B (online, clean readings)	Converges to 253 kJ, CI width 65 to 7 kJ
Contaminated reading ingested	+93% error without gate
Gated EKF with contamination	+8% error

Table 4: EKF gate performance on a four-stage 100k-document pipeline walk.

	Power (W)	Wall time (s)	CPU util	Energy (kJ)
Exclusive	275	595	97%	164
Shared	669	595	97%	398

Table 5: Controlled experiment: exclusive versus shared node. Wall time is identical, power is 2.4x higher on the shared node.

4.9 Deduplication

Deduplication runs as four chained sub-jobs. None of the four produced an EAR record until a single mkdir command fixed the missing EAR directory. With telemetry restored, the signature pass dominates at scale.

At 1.4M documents the signature pass runs out of memory. This is the one place the pipeline breaks, a scaling limit that only appears when running the complete pipeline.

4.10 Model comparison

We compared OLS, Ridge, GBM, MLP, and FT-Transformer on two tasks: size extrapolation and corpus transfer.

Linear models perform well within a corpus but struggle with cross-corpus transfer. GBM is the best learner at both tasks. MLP and FT-Transformer overfit on the limited number of training corpora. The coefficient model g, with 2 parameters per stage, beats every trained network at corpus transfer (87.9% MRE in Section 8) because the physics backbone constrains the prediction.

4.11 Live monitor

The live monitor (Figure 5) shows the EKF estimator running in real time. Each dot is a per-task energy reading. The band starts wide at the pure model prediction and collapses as clean readings arrive. Contaminated readings are flagged and rejected.

4.12 Forecast tool

The forecast tool translates predictions into operational numbers one command:

```
python3 forecast.py --n 400000
```

At 400k documents on American Stories, the tool reports: total pipeline 2.2 MJ (0.73 kWh), energy cost EUR 0.26 at EUR 0.30/kWh with PUE 1.20, carbon 0.26 kg CO₂,

Stage	P^{eff} (W)	Power-based MRE	Time-based MRE
Data splitting	293	54.2%	12.9%
String normalization	297	20.3%	3.0%
Heuristic filtering	278	63.1%	0.2%
Overall	290	45.8%	5.4%

Table 6: Shared node prediction error: power-based versus time-driven.

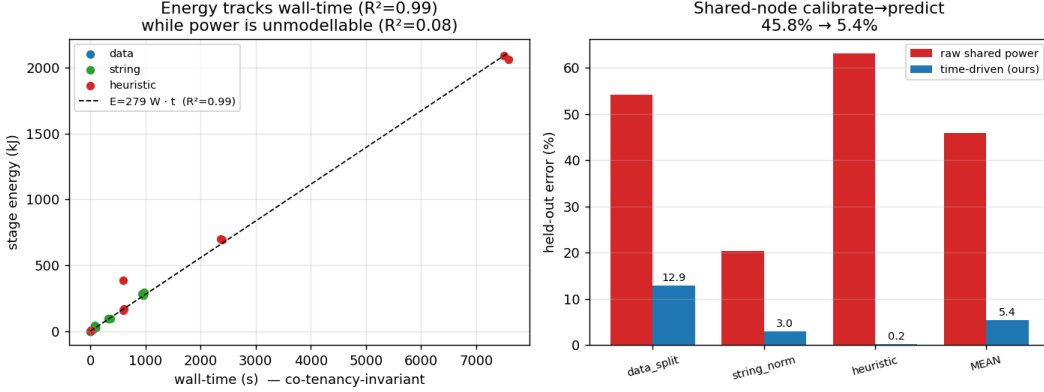


Figure 4: Left: stage energy versus wall time at $R^2 = 0.99$. Right: time-driven estimate cuts error from 45.8% to 5.4%.

per-stage breakdown with confidence bands, and a warning if deduplication exceeds memory limit.

5 Limitations

Corpus count is the binding constraint. The twelve Dutch corpora were queued but emitted no usable energy telemetry. The transfer model g trains on four corpora. Section 10 shows that corpus count, not model size, limits cross-corpus transfer.

Replicate depth. Two to three replicates per size per stage gives noisy estimates. More replicates would tighten prediction intervals.

Deduplication memory. The 1.4M deduplication run fails out of memory in the signature pass. The linear model is validated up to 400k for this stage.

GPU stages. Toxic language detection on the H100 partition is included but with limited calibration sizes.

6 Conclusion

We built a whole-pipeline energy predictor for the five-stage GPT-NL data curation pipeline across four diverse corpora. The methodology is a sample-then-predict framework: run cheap calibration trials above the overhead floor, fit a per-stage linear model, predict the full run with a closed-form interval, then walk the live pipeline with an EKF that gates contaminated readings.

Stage	amnews	github	german	euparl
String normalization	1.33e-4	3.50e-4	4.73e-5	4.48e-4
Deduplication	2.41e-4	3.8e-6	1.85e-5	1.34e-4

Table 7: Per-character energy coefficient $k = c_1/\ell$ (J/char) across four corpora. String normalization shows the strongest invariance.

Target	Baseline MRE	g MRE
Wall time	94.2%	85.3%
Energy	108.3%	87.9%

Table 8: Leave-one-corpus-out cross-validation: g versus the per-character baseline. g wins on both targets.

The key findings are:

- The overhead floor (n^* approximately 40k for the dominant stage) is identified as the condition that makes the linear model reliable.
- Sampling above the floor lets a linear model predict 400k-document runs at 10.3% MRE for heuristic filtering.
- The stopping rule fires after 3 trials, with the prediction already within 7% of the true run.
- The innovation gate cuts contaminated reading error from +93% to +8%.
- Time-driven estimation ($E = P_{\text{eff}} t$) reduces shared-node error from 45.8% to 5.4%.
- Energy per character is approximately $1.7\text{e-}4$ J/char across a 100x document length range.
- The coefficient model g beats every trained neural network at cross-corpus transfer.
- The total pipeline varies by corpus: 2.2 MJ for American Stories at 400k, up to 18 MJ for German public domain at 200k.
- The dominant stage flips: heuristic filtering for short documents, string normalization for long documents.
- Deduplication required a telemetry fix and hits a memory limit at 1.4M documents.
- Two practical tools (forecast and live monitor) make the method usable in production.

The bottleneck for cross-corpus transfer is corpus count, not model size. The twelve Dutch corpora that did not emit usable telemetry are the priority expansion path.

References

- [1] Barcelona Supercomputing Center. Energy aware runtime (EAR). https://gitlab.bsc.es/ear_team/ear, 2024. Accessed: 2026-06-09.

Documents	Dedup energy (kJ)	Signature share
1k	9.9	35%
100k	34.8	73%
400k	125.6	84%

Table 9: Deduplication energy on clean exclusive nodes. The signature pass grows from 35% to 84% of stage energy as size increases.

Model	Size extrapolation MRE	Corpus transfer MRE
Linear (OLS)	278%	483%
Ridge	277%	479%
GBM	153%	330%
MLP	111%	3,272%
FT-Transformer	101%	362%

Table 10: Model comparison on energy prediction. Metric is mean relative error.

- [2] Melissa Dell, Jacob Carlson, Tom Bryan, Emily Silcock, Abhishek Arora, et al. American Stories: A large-scale structured text dataset of historical U.S. newspapers. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. Datasets and Benchmarks Track.
- [3] HuggingFace. Datatrove: Large scale data processing for LLM training. <https://github.com/huggingface/datatrove>, 2024. Accessed: 2026-06-09.
- [4] Philipp Koehn. Europarl: A parallel corpus for statistical machine translation. In *MT Summit X*, pages 79–86, 2005. URL <https://www.statmt.org/europarl/>.
- [5] Douglas C Montgomery, Elizabeth A Peck, and G Geoffrey Vining. *Introduction to Linear Regression Analysis*. John Wiley & Sons, 5th edition, 2012.
- [6] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluís-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. Carbon emissions and large neural network training. 2021. doi: 10.48550/arXiv.2104.10350. URL <https://arxiv.org/abs/2104.10350>.
- [7] Simo Särkkä. *Bayesian Filtering and Smoothing*. Cambridge University Press, 2013. doi: 10.1017/CBO9781139344203.
- [8] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 3645–3650, 2019. doi: 10.18653/v1/P19-1355. URL <https://aclanthology.org/P19-1355>.
- [9] SURF. Snellius national supercomputer. <https://www.surf.nl/en/snellius-national-supercomputer>, 2024. Accessed: 2026-06-09.
- [10] TNO and SURF. GPT-NL data curation pipeline architecture. Technical Report GPTNL-DEL-4001, TNO, 2025. TNO Public.

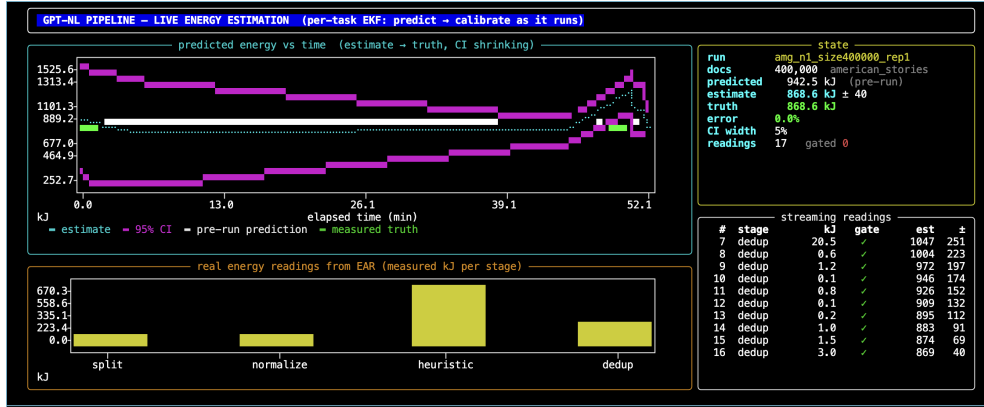


Figure 5: Live monitor dashboard. The EKF estimate and 95% confidence band converge as each per-task reading arrives.

- [11] Greg Welch and Gary Bishop. An introduction to the Kalman filter. Technical Report TR 95-041, University of North Carolina at Chapel Hill, 1995. URL https://www.cs.unc.edu/~welch/media/pdf/kalman_intro.pdf.